



A fast compact exponential time differencing method for semilinear parabolic equations with Neumann boundary conditions[☆]



Jianguo Huang^a, Lili Ju^{b,c,*}, Bo Wu^a

^a School of Mathematical Sciences, and MOE-LSC, Shanghai Jiao Tong University, Shanghai 200240, China

^b Department of Mathematics, University of South Carolina, Columbia, SC 29208, USA

^c School of Mathematical Sciences, Ocean University of China, Qingdao, Shandong 266100, China

ARTICLE INFO

Article history:

Received 28 January 2019

Received in revised form 6 March 2019

Accepted 6 March 2019

Available online 14 March 2019

Keywords:

Semilinear parabolic equations

Compact difference scheme

Neumann boundary condition

Exponential integrator

Fast Fourier transform

ABSTRACT

In this paper we propose a fast compact exponential time differencing method for solving a class of semilinear parabolic equations with Neumann boundary conditions. The model equation is first discretized in space by a fourth-order compact finite difference scheme with an appropriate treatment of the boundary condition, the resulting semi-discretized system is then diagonalized with fast Fourier transforms, and further expressed in a temporal integral formulation by the use of exponential integrators according to the Duhamel principle. The fully discrete scheme is finally obtained by using multistep interpolations for the nonlinear terms and exact evaluations of the underlying integrals. Some numerical experiments are performed to demonstrate the accuracy and efficiency of the proposed method.

© 2019 Elsevier Ltd. All rights reserved.

1. Introduction

Semilinear parabolic equations are related to many mathematical models arising from various scientific and engineering applications; we refer to [1,2] and references therein for details. In this paper, we are concerned with designing efficient and accurate numerical methods for solution of the following semilinear parabolic equation:

$$\begin{cases} u_t = D\Delta u - f(u), & \mathbf{x} \in \Omega, t > 0, & (a) \\ u(\mathbf{x}, 0) = u_0(\mathbf{x}), & \mathbf{x} \in \Omega, & (b) \end{cases} \quad (1.1)$$

where Ω is an open rectangular domain in $\mathbb{R}^d$ ($d = 2, 3$), $T > 0$ is the duration time, $D > 0$ is the diffusion coefficient, and $f(u)$ is the reaction term and stands for a Lipschitz continuous potential function.

[☆] This work was partially supported by National Natural Science Foundation of China under Grant numbers 11571237 and 11871454, and US National Science Foundation under grant number DMS-1818438.

* Corresponding author at: Department of Mathematics, University of South Carolina, Columbia, SC 29208, USA.

E-mail addresses: jghuang@sjtu.edu.cn (J. Huang), ju@math.sc.edu (L. Ju), sanshiyayan@sjtu.edu.cn (B. Wu).

In the past few decades, much work has been done for numerical solution of the problem (1.1). In the cases of Dirichlet or periodic boundary conditions, a class of efficient numerical methods have been developed based on the spatial discretization by compact difference schemes (CDS) (cf. [3,4]) and the temporal integration with exponential integrator based methods, including the integrating factor (IF) method (cf. [5–10]) and the exponential time differencing (ETD) method (cf. [11–19]). As a matter of fact, the CDS schemes have been well-used in computational dynamics, electromagnetic and computational acoustic due to its compact stencils and high accuracy, and in addition, they are also very suitable for handling Dirichlet and periodic boundary conditions by imposing function values at boundary nodes directly. The classic IF methods often apply numerical quadrature rules directly to the whole integrand, while the ETD methods first approximate the nonlinear terms by Lagrange interpolations and then carry out exact integration of the resulting integrals. The ETD methods are usually more accurate than the IF ones for highly stiff systems due to highly different decaying rates of the exponential integrator components. In [1], a fast numerical method was proposed for efficiently solving (1.1) by using the classic second-order central difference scheme on rectangular meshes combined with an ETD multistep approach and fast Fourier transform (FFT) based fast algorithms. The remarkable advantage is that this method can be used to handle various homogeneous and inhomogeneous boundary conditions in a unified way. Later, in [2], a fourth-order CDS scheme for spatial discretization and high-order ETD Runge–Kutta approach for temporal discretization was successfully integrated together to produce a fast high-order accurate method for Eq. (1.1) with Dirichlet or periodic boundary conditions, but unfortunately such treatment cannot be applied to the case of Neumann boundary condition. Note that although there already exist some researches on studying CDS schemes for the parabolic problem with Neumann boundary condition (cf. [20–25]), almost none of them ever considers whether FFT-based fast implementation can be applied to the resulting discrete problems, especially for high-order CDS schemes.

In this paper, we intend to resolve the above open problem by developing a fast compact ETD method for solving (1.1) with Neumann boundary condition. The spatial discretization is particularly performed with a CDS scheme as follows: for the interior grid nodes, we discretize the equation with fourth-order accuracy in the standard way as done in [2], while for the boundary grid nodes, we carry out the spatial discretization based on the approach proposed in [24], so as to get a semi-discretization of Eq. (1.1) with fourth-order accuracy in space globally. The advantage of such treatment is that not only the scheme enjoys the desired high accuracy but also the related matrices $\mathbf{A}_x$, $\mathbf{B}_x$, $\mathbf{A}_y$ and $\mathbf{B}_y$ appeared in (2.15) are all tridiagonal which make it possible to diagonalize the semi-discrete system. Our key observation is that the two matrices $\mathbf{A}_x^{-1}\mathbf{B}_x$ and $\mathbf{A}_y^{-1}\mathbf{B}_y$ in (2.15) are indeed diagonalizable, and the eigenvector matrices $\mathbf{P}_x$ and $\mathbf{P}_y$ are the same as those in [1, p. 441], thus we are able to develop FFT-based fast implementation following the way in [1,26]. To this end, we express the solution of the semi-discrete system (2.15) in an integral formulation with the use of exponential integrators by means of the Duhamel principle and all matrix multiplications could be implemented by discrete Fourier transform (DFT) or inverse discrete Fourier transform (iDFT).

2. A fast compact exponential time differencing method in two dimensions

2.1. Spatial discretization: fourth-order compact finite difference

Similar to [13], we first reformulate Eq. (1.1) as

$$\frac{\partial u}{\partial t} = (D\Delta - \kappa)u - \hat{f}(u), \quad (2.1)$$

where $\kappa \geq 0$ is a constant stabilizing parameter such that $\kappa \geq \frac{1}{2} \max\{0, \max_u f'(u)\}$ and $\hat{f}(u) := f(u) - \kappa u$. It is assumed that $f'(u)$ has an upper bound. For simplicity of presentation, we consider the two dimensional case and note that the extension of the proposed method to three dimensional problems is very natural. Let

$\Omega := \{x_b < x < x_e, y_b < y < y_e\}$ denote the target spatial domain with the Neumann boundary condition being imposed:

$$\begin{cases} \frac{\partial u}{\partial x}(x, y, t) = b^x(x, y, t), & x = x_b \text{ or } x = x_e, y_b \leq y \leq y_e, t > 0, \\ \frac{\partial u}{\partial y}(x, y, t) = b^y(x, y, t), & y = y_b \text{ or } y = y_e, x_b \leq x \leq x_e, t > 0. \end{cases} \quad (2.2)$$

We uniformly partition the domain Ω by a rectangular mesh, with the mesh sizes in x and y directions given by $h_x = \frac{x_e - x_b}{N_x}$ and $h_y = \frac{y_e - y_b}{N_y}$, respectively, and the grid points $(x_i, y_j) = (x_b + ih_x, y_b + jh_y)$. Denote the semi-discretize (in space) solution by $u_{i,j} = u_{i,j}(t) \approx u(x_i, y_j, t)$ for $0 \leq i \leq N_x$ and $0 \leq j \leq N_y$. Similarly, denote by $u_{i,j}^{xx}$ (reps. $u_{i,j}^{yy}$) the semi-discrete approximation of the second-order partial derivatives $u_{xx}(x_i, y_j, t)$ (reps. $u_{yy}(x_i, y_j, t)$). Next we will construct a CDS scheme with fourth-order accuracy for the spatial discretization of (1.1). For the interior point (x_i, y_j) , $1 \leq i \leq N_x - 1, 1 \leq j \leq N_y - 1$, as given in [4], the finite difference scheme can be expressed as

$$\begin{cases} \frac{1}{12}(u_{i-1,j}^{xx} + 10u_{i,j}^{xx} + u_{i+1,j}^{xx}) = \frac{1}{h_x^2}(u_{i-1,j} - 2u_{i,j} + u_{i+1,j}), \\ \frac{1}{12}(u_{i,j-1}^{yy} + 10u_{i,j}^{yy} + u_{i,j+1}^{yy}) = \frac{1}{h_y^2}(u_{i,j-1} - 2u_{i,j} + u_{i,j+1}). \end{cases} \quad (2.3)$$

In order to develop an appropriate fourth-order finite difference scheme for the boundary points, we first cite a useful result described as follows.

Lemma 2.1 ([24]). Assume that $f(x) \in C^5(x_b, x_e)$. Then

$$f'(x_0) = \frac{f(x_1) - f(x_0)}{h} - \frac{5}{12}hf''(x_0) - \frac{1}{12}hf''(x_1) - \frac{1}{12}h^2f^{(3)}(x_0) + \mathcal{O}(h^4), \quad (2.4)$$

$$f'(x_{N_x}) = \frac{f(x_{N_x}) - f(x_{N_x-1})}{h} + \frac{5}{12}hf''(x_{N_x}) + \frac{1}{12}hf''(x_{N_x-1}) - \frac{1}{12}h^2f^{(3)}(x_{N_x}) + \mathcal{O}(h^4). \quad (2.5)$$

The above lemma concludes that for deriving a high order scheme on the boundary node for the one-dimensional problem, we need to deal with the second and third derivatives of the solution. This observation also applies to two and three-dimensional problems. Taking the first-order partial differentiation of Eq. (1.1) with respect to x gives

$$u_{tx} - Du_{xxx} - Du_{yyx} = f'(u)u_x.$$

Letting $x \rightarrow x_b +$ and $x \rightarrow x_e -$, respectively, we have by virtue of the boundary condition (2.2) that

$$\begin{aligned} \frac{\partial^3 u}{\partial x^3} \Big|_{0,j} &= \frac{1}{D} \frac{\partial b^x}{\partial t} \Big|_{0,j} - \frac{\partial^2 b^x}{\partial y^2} \Big|_{0,j} + \frac{1}{D} \left(\frac{\partial f}{\partial u} b^x \right)_{0,j}, \\ \frac{\partial^3 u}{\partial x^3} \Big|_{N_x,j} &= \frac{1}{D} \frac{\partial b^x}{\partial t} \Big|_{N_x,j} - \frac{\partial^2 b^x}{\partial y^2} \Big|_{N_x,j} + \frac{1}{D} \left(\frac{\partial f}{\partial u} b^x \right)_{N_x,j}. \end{aligned} \quad (2.6)$$

Similarly, we can find that

$$\begin{aligned} \frac{\partial^3 u}{\partial y^3} \Big|_{i,0} &= \frac{1}{D} \frac{\partial b^y}{\partial t} \Big|_{i,0} - \frac{\partial^2 b^y}{\partial x^2} \Big|_{i,0} + \frac{1}{D} \left(\frac{\partial f}{\partial u} b^y \right)_{i,0}, \\ \frac{\partial^3 u}{\partial y^3} \Big|_{i,N_y} &= \frac{1}{D} \frac{\partial b^y}{\partial t} \Big|_{i,N_y} - \frac{\partial^2 b^y}{\partial x^2} \Big|_{i,N_y} + \frac{1}{D} \left(\frac{\partial f}{\partial u} b^y \right)_{i,N_y}. \end{aligned} \quad (2.7)$$

On the other hand, according to Lemma 2.1 it follows that

$$\begin{aligned}\frac{\partial u(x_0, y, t)}{\partial x} &= \frac{u(x_1, y, t) - u(x_0, y, t)}{h_x} - \frac{5}{12} h_x \frac{\partial^2 u(x_0, y, t)}{\partial x^2} \\ &\quad - \frac{1}{12} h_x \frac{\partial^2 u(x_1, y, t)}{\partial x^2} - \frac{1}{12} h_x^2 \frac{\partial^3 u(x_0, y, t)}{\partial x^3} + \mathcal{O}(h_x^4), \\ \frac{\partial u(x_{N_x}, y, t)}{\partial x} &= \frac{u(x_{N_x}, y, t) - u(x_{N_x-1}, y, t)}{h_x} + \frac{5}{12} h_x \frac{\partial^2 u(x_{N_x}, y, t)}{\partial x^2} \\ &\quad + \frac{1}{12} h_x \frac{\partial^2 u(x_{N_x-1}, y, t)}{\partial x^2} - \frac{1}{12} h_x^2 \frac{\partial^3 u(x_{N_x}, y, t)}{\partial x^3} + \mathcal{O}(h_x^4).\end{aligned}$$

Ignoring the higher order quantities gives rise to

$$\begin{cases} \frac{1}{12} (5u_{0,j}^{xx} + u_{1,j}^{xx}) = \frac{1}{h_x^2} (u_{1,j} - u_{0,j}) - \frac{1}{h_x} (b^x)_{0,j} - \frac{h_x}{12} \frac{\partial^3 u}{\partial x^3} \Big|_{0,j}, \\ \frac{1}{12} (5u_{N_x,j}^{xx} + u_{N_x-1,j}^{xx}) = -\frac{1}{h_x^2} (u_{N_x,j} - u_{N_x-1,j}) + \frac{1}{h_x} (b^x)_{N_x,j} + \frac{h_x}{12} \frac{\partial^3 u}{\partial x^3} \Big|_{N_x,j}. \end{cases} \quad (2.8)$$

Similarly, we also have

$$\begin{cases} \frac{1}{12} (5u_{i,0}^{yy} + u_{i,1}^{yy}) = \frac{1}{h_y^2} (u_{i,1} - u_{i,0}) - \frac{1}{h_y} (b^y)_{i,0} - \frac{h_y}{12} \frac{\partial^3 u}{\partial y^3} \Big|_{i,0}, \\ \frac{1}{12} (5u_{i,N_y}^{yy} + u_{i,N_y-1}^{yy}) = -\frac{1}{h_y^2} (u_{i,N_y} - u_{i,N_y-1}) + \frac{1}{h_y} (b^y)_{i,N_y} + \frac{h_y}{12} \frac{\partial^3 u}{\partial y^3} \Big|_{i,N_y}. \end{cases} \quad (2.9)$$

Inserting (2.6) and (2.7) into Eqs. (2.8) and (2.9), we arrive at

$$\begin{cases} \frac{1}{12} (5u_{0,j}^{xx} + u_{1,j}^{xx}) = \frac{1}{h_x^2} (u_{1,j} - u_{0,j}) - \frac{1}{h_x} (b^x)_{0,j} - \frac{h_x}{12} \left(\frac{1}{D} \frac{\partial b^x}{\partial t} \Big|_{0,j} - \frac{\partial^2 b^x}{\partial y^2} \Big|_{0,j} + \frac{1}{D} \left(\frac{\partial f}{\partial u} b^x \right)_{0,j} \right), \\ \frac{1}{12} (5u_{N_x,j}^{xx} + u_{N_x-1,j}^{xx}) = -\frac{1}{h_x^2} (u_{N_x,j} - u_{N_x-1,j}) + \frac{1}{h_x} (b^x)_{N_x,j} \\ \quad + \frac{h_x}{12} \left(\frac{1}{D} \frac{\partial b^x}{\partial t} \Big|_{N_x,j} - \frac{\partial^2 b^x}{\partial y^2} \Big|_{N_x,j} + \frac{1}{D} \left(\frac{\partial f}{\partial u} b^x \right)_{N_x,j} \right), \\ \frac{1}{12} (5u_{i,0}^{yy} + u_{i,1}^{yy}) = \frac{1}{h_y^2} (u_{i,1} - u_{i,0}) - \frac{1}{h_y} (b^y)_{i,0} - \frac{h_y}{12} \left(\frac{1}{D} \frac{\partial b^y}{\partial t} \Big|_{i,0} - \frac{\partial^2 b^y}{\partial x^2} \Big|_{i,0} + \frac{1}{D} \left(\frac{\partial f}{\partial u} b^y \right)_{i,0} \right), \\ \frac{1}{12} (5u_{i,N_y}^{yy} + u_{i,N_y-1}^{yy}) = -\frac{1}{h_y^2} (u_{i,N_y} - u_{i,N_y-1}) + \frac{1}{h_y} (b^y)_{i,N_y} \\ \quad + \frac{h_y}{12} \left(\frac{1}{D} \frac{\partial b^y}{\partial t} \Big|_{i,N_y} - \frac{\partial^2 b^y}{\partial x^2} \Big|_{i,N_y} + \frac{1}{D} \left(\frac{\partial f}{\partial u} b^y \right)_{i,N_y} \right). \end{cases} \quad (2.10)$$

Next we would like to formulate the above difference scheme in matrix notation. Define

$$\mathbf{A}_x = \mathbf{R}_{(N_x+1) \times (N_x+1)}, \quad \mathbf{A}_y = \mathbf{R}_{(N_y+1) \times (N_y+1)}, \quad \mathbf{B}_x = -\mathbf{G}_{(N_x+1) \times (N_x+1)}, \quad \mathbf{B}_y = -\mathbf{G}_{(N_y+1) \times (N_y+1)},$$

where, for a natural number p ,

$$\mathbf{R}_{p \times p} := \begin{bmatrix} 10 & 2 & & & \\ 1 & 10 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & 10 & 1 \\ & & & 2 & 10 \end{bmatrix}_{p \times p}, \quad \mathbf{G}_{p \times p} := \begin{bmatrix} -2 & 2 & & & \\ 1 & -2 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & -2 & 1 \\ & & & 2 & -2 \end{bmatrix}_{p \times p}. \quad (2.11)$$

Then, the combination of (2.3) and (2.10) yields the matrix (or two-dimensional array) form of equations:

$$\mathbf{A}_x \mathbf{U}^{xx} = -\frac{12}{h_x^2} \mathbf{B}_x \mathbf{U} + \frac{1}{D} \left(\mathcal{W}^x + \mathcal{F}_1(\mathbf{U}) \right), \quad \mathbf{U}^{yy} \mathbf{A}_y^T = -\frac{12}{h_y^2} \mathbf{U} \mathbf{B}_y^T + \frac{1}{D} \left(\mathcal{W}^y + \mathcal{F}_2(\mathbf{U}) \right), \quad (2.12)$$

where $\mathbf{U} = (u_{i,j})$, $\mathbf{U}^{xx} = (u_{i,j}^{xx})$, $\mathbf{U}^{yy} = (u_{i,j}^{yy})$, and $\mathcal{F}_1(\mathbf{U}) = (f_{i,j}^1)$, $\mathcal{F}_2(\mathbf{U}) = (f_{i,j}^2)$, $\mathcal{W}^x = (w_{i,j}^x)$, $\mathcal{W}^y = (w_{i,j}^y)$ are defined respectively by

$$f_{i,j}^1 = \begin{cases} -2h_x f'(u_{0,j}) b_{0,j}^x, & i = 0, \quad 0 \leq j \leq N_y, \\ 2h_x f'(u_{N_x,j}) b_{N_x,j}^x, & i = N_x, \quad 0 \leq j \leq N_y, \\ 0, & \text{otherwise,} \end{cases}$$

$$f_{i,j}^2 = \begin{cases} -2h_y f'(u_{i,0}) b_{i,0}^y, & j = 0, \quad 0 \leq i \leq N_x, \\ 2h_y f'(u_{i,N_y}) b_{i,N_y}^y, & j = N_y, \quad 0 \leq i \leq N_x, \\ 0, & \text{otherwise,} \end{cases}$$

$$w_{i,j}^x = \begin{cases} -\frac{24D}{h_x} b_{0,j}^x - 2h_x \left(\frac{\partial b^x}{\partial t} \Big|_{0,j} - D \frac{\partial^2 b^x}{\partial y^2} \Big|_{0,j} \right), & i = 0, \quad 0 \leq j \leq N_y, \\ \frac{24D}{h_x} b_{N_x,j}^x + 2h_x \left(\frac{\partial b^x}{\partial t} \Big|_{N_x,j} - D \frac{\partial^2 b^x}{\partial y^2} \Big|_{N_x,j} \right), & i = N_x, \quad 0 \leq j \leq N_y, \\ 0 & \text{otherwise,} \end{cases} \quad (2.13)$$

$$w_{i,j}^y = \begin{cases} -\frac{24D}{h_y} b_{i,0}^y - 2h_y \left(\frac{\partial b^y}{\partial t} \Big|_{i,0} - D \frac{\partial^2 b^y}{\partial x^2} \Big|_{i,0} \right), & j = 0, \quad 0 \leq i \leq N_x, \\ \frac{24D}{h_y} b_{i,N_y}^y + 2h_y \left(\frac{\partial b^y}{\partial t} \Big|_{i,N_y} - D \frac{\partial^2 b^y}{\partial x^2} \Big|_{i,N_y} \right), & j = N_y, \quad 0 \leq i \leq N_x, \\ 0 & \text{otherwise.} \end{cases} \quad (2.14)$$

Thus we obtain a semi-discretization with fourth-order spatial accuracy (an ODE system) for the model equation (1.1) as follows:

$$\frac{d\mathbf{U}}{dt} + \left(\frac{12D}{h_x^2} \mathbf{A}_x^{-1} \mathbf{B}_x \mathbf{U} + \frac{12D}{h_y^2} \mathbf{U} (\mathbf{A}_y^{-1} \mathbf{B}_y)^T + \kappa \mathbf{U} \right) = \mathbf{G}(t, \mathbf{U}) := -\mathcal{F}(\mathbf{U}) + \mathbf{W}(t), \quad (2.15)$$

where $\mathcal{F}(\mathbf{U}) := \hat{f}(\mathbf{U}) - \mathbf{A}_x^{-1} \mathcal{F}_1(\mathbf{U}) - \mathcal{F}_2(\mathbf{U}) (\mathbf{A}_y^T)^{-1}$ and $\mathbf{W} := \mathbf{A}_x^{-1} \mathcal{W}^x + \mathcal{W}^y (\mathbf{A}_y^T)^{-1}$.

2.2. Exponential time integration and fast calculation

In this subsection, we will devise a fast implementation of (2.15) based on FFT. At first glance, it seems impossible to achieve the purpose since (2.15) involves the matrices $\mathbf{A}_x^{-1} \mathbf{B}_x$ and $\mathbf{A}_y^{-1} \mathbf{B}_y$ which are not symmetric and difficult to handle. A useful observation is that the two matrices are actually diagonalizable with the similarity matrices $\mathbf{P}_x$ and $\mathbf{P}_y$, the same ones as those in [1, p. 441]. Hence, we are able to develop a fast implementation following the ideas in [1, 26]. Let $\mathbf{P}_x$ be the non-singular matrix given in [1, p. 441] such that

$$\mathbf{G}_{(N_x+1) \times (N_x+1)} = \mathbf{P}_x \mathbf{A}_x \mathbf{P}_x^{-1},$$

where $\mathbf{A}_x := \text{diag}(d_0^x, d_1^x, \dots, d_{N_x}^x)$ and $d_k^x = -4 \sin^2 \left(\frac{k\pi}{2N_x} \right)$ for $k = 0, 1, \dots, N_x$. Then

$$\mathbf{A}_x = 12\mathbf{I} + \mathbf{G}_{(N_x+1) \times (N_x+1)} = \mathbf{P}_x (12\mathbf{I} + \mathbf{A}_x) \mathbf{P}_x^{-1}, \quad \mathbf{B}_x = -\mathbf{G}_{(N_x+1) \times (N_x+1)} = -\mathbf{P}_x \mathbf{A}_x \mathbf{P}_x^{-1},$$

where $\mathbf{I}$ is an identity matrix. Similarly, we also have

$$\mathbf{A}_y = \mathbf{P}_y (12\mathbf{I} + \mathbf{A}_y) \mathbf{P}_y^{-1}, \quad \mathbf{B}_y = -\mathbf{P}_y \mathbf{A}_y \mathbf{P}_y^{-1},$$

where $\mathbf{A}_y := \text{diag}(d_0^y, d_1^y, \dots, d_{N_y}^y)$ and $d_k^y = -4 \sin^2\left(\frac{k\pi}{2N_y}\right)$ for $k = 0, 1, \dots, N_y$. Thus the ODE system (2.15) can be rewritten as

$$\frac{d\mathbf{U}}{dt} + \left(\tilde{\mathbf{A}}_x \mathbf{U} + \mathbf{U} \tilde{\mathbf{B}}_y^T + \kappa \mathbf{U} \right) = \mathbf{G}, \quad (2.16)$$

where

$$\tilde{\mathbf{A}}_x = -\frac{12D}{h_x^2} \mathbf{P}_x (12\mathbf{I} + \mathbf{A}_x)^{-1} \mathbf{A}_x \mathbf{P}_x^{-1}, \quad \tilde{\mathbf{B}}_y = -\frac{12D}{h_y^2} \mathbf{P}_y (12\mathbf{I} + \mathbf{A}_y)^{-1} \mathbf{A}_y \mathbf{P}_y^{-1}.$$

Define two special operators $\otimes$ and $\odot$ related respectively to $\mathbf{C}_x \in \mathbb{R}^{(N_x+1) \times (N_x+1)}$ and $\mathbf{C}_y \in \mathbb{R}^{(N_y+1) \times (N_y+1)}$ by (cf. [1])

$$(\mathbf{C}_x \otimes \mathbf{U})_{i,j} = \sum_{l=0}^{N_x} (\mathbf{C}_x)_{i,l} u_{l,j}, \quad (\mathbf{C}_y \odot \mathbf{U})_{i,j} = \sum_{l=0}^{N_y} (\mathbf{C}_y)_{j,l} u_{i,l}. \quad (2.17)$$

Let $\mathbf{v} = [v_0, v_1, \dots, v_{N_x}]^T$ be any vector of length $N_x + 1$ and correspondingly define its reflection vector of length $2N_x$ by

$$\hat{\mathbf{v}} = [v_0, v_1, \dots, v_{N_x}, v_{N_x-1}, \dots, v_1]^T.$$

Then, as given in [1], for a vector $\mathbf{v} \in \mathbb{R}^{N_x+1}$, define the operator “RDFT” (resp. “iRDFT”) on $\mathbf{v}$ as to applying a DFT (resp. an inverse DFT) to $\hat{\mathbf{v}}$ and then taking the first $N_x + 1$ components of the result. Moreover, for a matrix $\mathbf{U}$ with $(N_x + 1)$ -rows, let RDFT($\mathbf{U}$) (reps. iRDFT($\mathbf{U}$)) be a matrix produced by applying a RDFT (resp. iRDFT) on each column of $\mathbf{U}$. It is proven that in the two dimensional case

$$\mathbf{P}_x \mathbf{U} = \mathbf{P}_x \otimes \mathbf{U} = \text{RDFT}(\mathbf{U}), \quad \mathbf{P}_x^{-1} \mathbf{U} = \mathbf{P}_x^{-1} \otimes \mathbf{U} = \text{iRDFT}(\mathbf{U}).$$

Similarly, for a matrix $\mathbf{V}$ with $(N_y + 1)$ -columns, we can easily derive that

$$\mathbf{V} \mathbf{P}_y^T = \mathbf{P}_y \odot \mathbf{V} = \text{RDFT}(\mathbf{V}^T)^T, \quad \mathbf{V} \mathbf{P}_y^{-T} = \mathbf{P}_y^{-1} \odot \mathbf{V} = \text{iRDFT}(\mathbf{V}^T)^T.$$

Let us define $\mathbf{H} = (h_{i,j})_{(N_x+1) \times (N_y+1)}$, $\mathbf{H}_1 = (h_{i,j}^1)_{(N_x+1) \times (N_y+1)}$ and $\mathbf{H}_2 = (h_{i,j}^2)_{(N_x+1) \times (N_y+1)}$ with

$$h_{i,j} = \frac{12Dd_i^x}{h_x^2(12 + d_i^x)} + \frac{12Dd_j^y}{h_y^2(12 + d_j^y)} - \kappa, \quad h_{i,j}^1 = \frac{1}{12 + d_i^x}, \quad h_{i,j}^2 = \frac{1}{12 + d_j^y}.$$

Introduce an operation “ (e^*) ” by taking exponentials of a matrix element by element as $(e^*)^{\mathbf{H}} = (e^{h_{i,j}})_{(N_x+1) \times (N_y+1)}$, and an operator “ $\odot$ ” for elementwise multiplication between two matrices of the same size as $(\mathbf{M} \odot \mathbf{L})_{i,j} = (m_{i,j} l_{i,j})$ for $\mathbf{M} = (m_{i,j})$ and $\mathbf{L} = (l_{i,j})$. By the Duhamel principle that the ODE system (2.15) can be reformulated as

$$\begin{aligned} \mathbf{U}_{m+1} &= \mathbf{P}_y \odot \mathbf{P}_x \otimes \left((e^*)^{\mathbf{H} \Delta t} \odot (\mathbf{P}_y^{-1} \odot \mathbf{P}_x^{-1} \otimes \mathbf{U}_m) \right. \\ &\quad \left. + \int_0^{\Delta t} (e^*)^{\mathbf{H}(\Delta t - \tau)} \odot \mathbf{P}_y^{-1} \odot \mathbf{P}_x^{-1} \otimes \left(-\mathcal{F}(\mathbf{U}(t_m + \tau)) + \mathbf{W}(t_m + \tau) \right) d\tau \right). \end{aligned} \quad (2.18)$$

where $\Delta t = t_{m+1} - t_m$ is the uniform time step size and $\mathbf{U}_{m+1} \approx \mathbf{U}(t_{m+1})$. For $0 \leq \tau \leq \Delta t$, let

$$\begin{aligned} \mathbf{F}(\mathbf{U}(t_m + \tau)) &:= (f_{i,j}(t_m + \tau))_{(N_x+1) \times (N_y+1)} = \mathbf{P}_y^{-1} \odot \mathbf{P}_x^{-1} \otimes \mathcal{F}(\mathbf{U}(t_m + \tau)) \\ &= \mathbf{P}_y^{-1} \odot \mathbf{P}_x^{-1} \otimes \hat{\mathbf{f}}(\mathbf{U}(t_m + \tau)) - \mathbf{H}_1 \odot \mathbf{P}_y^{-1} \odot \mathbf{P}_x^{-1} \otimes \mathcal{F}_1(\mathbf{U}(t_m + \tau)) \\ &\quad - \mathbf{H}_2 \odot \mathbf{P}_y^{-1} \odot \mathbf{P}_x^{-1} \otimes \mathcal{F}_2(\mathbf{U}(t_m + \tau)), \end{aligned}$$

$$\begin{aligned} \overline{\mathbf{W}}(t_m + \tau) &:= (\overline{w}_{i,j}(t_m + \tau))_{(N_x+1) \times (N_y+1)} = \mathbf{P}_y^{-1} \odot \mathbf{P}_x^{-1} \otimes \mathbf{W}(t_m + \tau) \\ &= \mathbf{H}_1 \odot \mathbf{P}_y^{-1} \odot \mathbf{P}_x^{-1} \otimes \mathcal{W}^x + \mathbf{H}_2 \odot \mathbf{P}_y^{-1} \odot \mathbf{P}_x^{-1} \otimes \mathcal{W}^y, \end{aligned}$$

$$\begin{aligned} \mathbf{Q}^F &:= (q_{i,j}^F)_{(N_x+1) \times (N_y+1)} = \int_0^{\Delta t} (e^*)^{H(\Delta t - \tau)} \odot \mathbf{F}(\mathbf{U}(t_m + \tau)) \, d\tau, \quad \mathbf{Q}^W \\ &:= (q_{i,j}^W)_{(N_x+1) \times (N_y+1)} = \int_0^{\Delta t} (e^*)^{H(\Delta t - \tau)} \odot \overline{\mathbf{W}}(t_m + \tau) \, d\tau. \end{aligned}$$

Then, the above equation (2.18) can be expressed as

$$\mathbf{U}_{m+1} = \mathbf{P}_y \odot \mathbf{P}_x \otimes \left((e^*)^{H\Delta t} \odot (\mathbf{P}_y^{-1} \odot \mathbf{P}_x^{-1} \otimes \mathbf{U}_m) + \mathbf{Q}^W - \mathbf{Q}^F \right). \quad (2.19)$$

Furthermore, by using multistep approximations of $\mathcal{F}(\mathbf{U})$ (with Adams–Bashforth scheme) and $\overline{\mathbf{W}}$ (with Adams–Moulton scheme) as done in [1], we finally obtain a compact ETD multistep method for Eq. (1.1) as

$$\mathbf{U}_{m+1} = \mathbf{P}_y \odot \mathbf{P}_x \otimes \left((e^*)^{H\Delta t} \odot (\mathbf{P}_y^{-1} \odot \mathbf{P}_x^{-1} \otimes \mathbf{U}_m) + \sum_{s=-1}^{r-1} \overline{\mathbf{W}}_{m-s} \odot \mathbf{S}_{r,s}^W - \sum_{s=0}^r \mathbf{F}_{m-s} \odot \mathbf{S}_{r,s}^F \right), \quad (2.20)$$

where r denotes the order of the Lagrange interpolating polynomials and

$$\mathbf{Q}^W \approx \sum_{s=-1}^{r-1} \overline{\mathbf{W}}_{m-s} \odot \mathbf{S}_{r,s}^W, \quad \mathbf{Q}^F \approx \sum_{s=0}^r \mathbf{F}_{m-s} \odot \mathbf{S}_{r,s}^F.$$

with $\mathbf{S}_{r,s}^W$ and $\mathbf{S}_{r,s}^F$ representing the exact integrations of the products of $(e^*)^{H(\Delta t - \tau)}$ with some Lagrange interpolating polynomials (see [1] for the details). The scheme (2.20) is fourth-order accurate in space and $(r+1)$ -th order accurate in time. Based on the FFT implementation, the overall complexity is $O(N^2 \log(N))$ per time step where $N = \max(N_x, N_y)$.

Remark 2.1. By following the approaches presented in [1,2], the above numerical method (2.20) can be straightforwardly extended to numerically solve Eq. (1.1) with Neumann boundary condition in three dimensions, through the use of three-dimensional array representations of the unknowns and equations.

3. Numerical experiments

In this section, some numerical examples are presented to illustrate the performance of the proposed fast compact ETD method (2.20). All numerical results are obtained using Matlab 2016b software on a laptop computer with an Intel(R) i7-8750H, 2.20 GHz CPU and 8 GB of memory.

Example 3.1. In this example we consider the problem

$$\begin{cases} \frac{\partial u}{\partial t} = \Delta u - u, & (x, y) \in \Omega = (-1, 1)^2, \, t \in (0, T], \\ u(x, y, 0) = \cos(x) \cos(y). \end{cases} \quad (3.1)$$

The exact solution is chosen to be $u(x, y, t) = \exp(-3t) \cos(x) \cos(y)$ and from which we impose the corresponding Neumann boundary condition.

Note that $f(u) = u$ and $f'(u) = 1$ in this example, thus we set the stabilizing parameter $\kappa = 1$. Numerical results on errors, convergence rates (CR) and CPU times at the final time $T = 1$ for Example 3.1 are reported in Table 1. For the spatial accuracy tests, we use the scheme of third-order temporal accuracy ($r = 2$) with a fixed time partition $N_t = 2048$. We can clearly observe the fourth-order convergence in space from the test results when the temporal errors are relatively negligible. For the temporal accuracy tests, we used a fixed spatial grid with $N_x = N_y = 1024$. We find as expected the first, second and third-order convergence in time for $r = 0, 1, 2$, respectively.

Table 1

Errors, convergence rates and CPU times (measured in seconds) at the final time $T = 1$ using the proposed method (2.20) in Example 3.1.

$(N_x \times N_y) \times N_t$	L_2 Error	CR	L_∞ Error	CR	CPU Time
Accuracy test of spatial discretization, $r = 2$					
$(4^2) \times 2048$	5.3914e-05	–	2.7349e-05	–	0.50
$(8^2) \times 2048$	2.9434e-06	4.20	1.7310e-06	3.98	0.52
$(16^2) \times 2048$	1.7019e-07	4.11	1.0872e-07	3.99	0.78
$(32^2) \times 2048$	1.0304e-08	4.05	6.8578e-09	3.99	1.54
$(64^2) \times 2048$	6.8146e-10	3.92	4.5229e-10	3.92	4.90
$(128^2) \times 2048$	7.7190e-11	3.14	4.3348e-11	3.38	12.38
Accuracy test of temporal discretization, $r = 0$					
$(1024^2) \times 8$	7.7859e-02	–	4.1182e-02	–	6.29
$(1024^2) \times 16$	4.0576e-02	0.94	2.1643e-02	0.93	12.10
$(1024^2) \times 32$	2.0715e-02	0.97	1.1118e-02	0.96	23.75
$(1024^2) \times 64$	1.0466e-02	0.98	5.6427e-03	0.98	47.07
$(1024^2) \times 128$	5.2605e-03	0.99	2.8452e-03	0.99	94.11
Accuracy test of temporal discretization, $r = 1$					
$(1024^2) \times 8$	5.2907e-03	–	2.8454e-03	–	6.56
$(1024^2) \times 16$	1.3222e-03	2.00	7.1344e-04	2.00	12.64
$(1024^2) \times 32$	3.3052e-04	2.00	1.7883e-04	2.00	24.86
$(1024^2) \times 64$	8.2628e-05	2.00	4.4799e-05	2.00	49.50
$(1024^2) \times 128$	2.0657e-05	2.00	1.1217e-05	2.00	97.77
Accuracy test of temporal discretization, $r = 2$					
$(1024^2) \times 8$	5.0011e-04	–	2.8767e-04	–	6.90
$(1024^2) \times 16$	6.4696e-05	2.95	3.6581e-05	2.98	13.40
$(1024^2) \times 32$	8.2420e-06	2.97	4.6269e-06	2.98	26.39
$(1024^2) \times 64$	1.0406e-06	2.99	5.8216e-07	2.99	52.34
$(1024^2) \times 128$	1.3074e-07	2.99	7.3025e-08	2.99	104.31

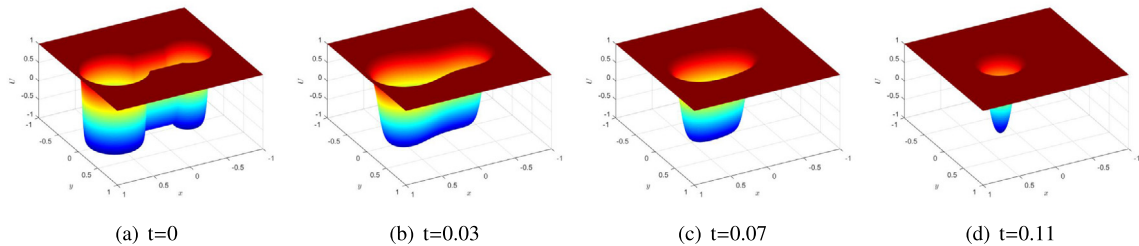


Fig. 1. Snapshots of the solution simulated by the third-order compact ETD scheme with $N_x = N_y = 1024$ and $\Delta t = 0.0005$ in Example 3.2.

Example 3.2. In this example, we solve the well-known Allen–Cahn equation:

$$\frac{\partial u}{\partial t} = \Delta u - \frac{1}{\epsilon^2}(u^3 - u), \quad (x, y) \in \Omega = (-1, 1)^2, \quad t > 0, \quad (3.2)$$

with homogeneous Neumann boundary condition. We set $\epsilon = 0.05$ and the initial configuration is set to be

$$u(x, y, 0) = \begin{cases} \tanh\left(\frac{3}{\epsilon}((x - 0.5)^2 + y^2 - (0.39)^2)\right), & x > 0.14, \\ \tanh\left(\frac{3}{\epsilon}(y^2 - (0.15)^2)\right), & -0.3 \leq x \leq 0.14, \\ \tanh\left(\frac{3}{\epsilon}((x + 0.5)^2 + y^2 - (0.25)^2)\right), & x < -0.3. \end{cases}$$

The stabilizing parameter is chosen to be $\kappa = \frac{2}{\epsilon^2}$ as suggested in [1]. Fig. 1 shows snapshots of the solution simulated by the third-order compact ETD scheme with $N_x = N_y = 1024$ and $\Delta t = 0.0005$. The numerical results match very well with those given in [27], and our scheme only took about 0.7963 s per time step.

References

- [1] L. Ju, J. Zhang, L. Zhu, Q. Du, Fast explicit integration factor methods for semilinear parabolic equations, *J. Sci. Comput.* 62 (2) (2015) 431–455.
- [2] L. Zhu, L. Ju, W. Zhao, Fast high-order compact exponential time differencing Runge-Kutta methods for second-order semilinear parabolic equations, *J. Sci. Comput.* 67 (3) (2016) 1043–1065.
- [3] S.K. Lele, Compact finite difference schemes with spectral-like resolution, *J. Comput. Phys.* 103 (1) (1992) 16–42.
- [4] M. Li, T. Tang, B. Fornberg, A compact fourth-order finite difference scheme for the steady incompressible Navier-Stokes equations, *Internat. J. Numer. Methods Fluids* 20 (10) (1995) 1137–1151.
- [5] T. Jiang, Y.-T. Zhang, Krylov single-step implicit integration factor WENO methods for advection-diffusion-reaction equations, *J. Comput. Phys.* 311 (2016) 22–44.
- [6] L. Ju, X.-F. Liu, W. Leng, Compact implicit integration factor methods for a family of semilinear fourth-order parabolic equations, *Discrete Contin. Dyn. Syst. Ser. B* 19 (6) (2014) 1667–1687.
- [7] J.D. Lawson, Generalized runge-kutta processes for stable systems with large lipschitz constants, *SIAM J. Numer. Anal.* 4 (3) (1967) 372–380.
- [8] Q. Nie, F.Y.M. Wan, Y.-T. Zhang, X.-F. Liu, Compact integration factor methods in high spatial dimensions, *J. Comput. Phys.* 227 (10) (2008) 5238–5255.
- [9] Q. Nie, Y.-T. Zhang, R. Zhao, Efficient semi-implicit schemes for stiff systems, *J. Comput. Phys.* 214 (2) (2006) 521–537.
- [10] D. Wang, L. Zhang, Q. Nie, Array-representation integration factor method for high-dimensional systems, *J. Comput. Phys.* 258 (2014) 585–600.
- [11] J. Certaine, The solution of ordinary differential equations with large time constants, in: *Mathematical Methods for Digital Computers*, Wiley, New York, 1960, pp. 128–132.
- [12] S.M. Cox, P.C. Matthews, Exponential time differencing for stiff systems, *J. Comput. Phys.* 176 (2) (2002) 430–455.
- [13] Q. Du, W. Zhu, Analysis and applications of the exponential time differencing schemes and their contour integration modifications, *BIT* 45 (2) (2005) 307–328.
- [14] M. Hochbruck, C. Lubich, H. Selhofer, Exponential integrators for large systems of differential equations, *SIAM J. Sci. Comput.* 19 (5) (1998) 1552–1574.
- [15] M. Hochbruck, A. Ostermann, Explicit exponential Runge-Kutta methods for semilinear parabolic problems, *SIAM J. Numer. Anal.* 43 (3) (2005) 1069–1090.
- [16] M. Hochbruck, A. Ostermann, Exponential integrators, *Acta Numer.* 19 (2010) 209–286.
- [17] E. Hairer, G. Wanner, Stiff and differential-algebraic problems, in: *Solving Ordinary Differential Equations. II*, second ed., in: *Springer Series in Computational Mathematics*, vol. 14, Springer-Verlag, Berlin, 1996.
- [18] S. Krogstad, Generalized integrating factor methods for stiff PDEs, *J. Comput. Phys.* 203 (1) (2005) 72–88.
- [19] D.A. Pope, An exponential method of numerical integration of ordinary differential equations, *Commun. ACM* 6 (1963) 491–493.
- [20] W. Liao, J. Zhu, A.Q.M. Khaliq, A fourth-order compact algorithm for nonlinear reaction-diffusion equations with Neumann boundary conditions, *Numer. Methods Partial Differential Equations* 22 (3) (2006) 600–616.
- [21] J. Zhao, W. Dai, T. Niu, Fourth-order compact schemes of a heat conduction problem with Neumann boundary conditions, *Numer. Methods Partial Differential Equations* 23 (5) (2007) 949–959.
- [22] W. Dai, An improved compact finite difference scheme for solving an N-carrier system with Neumann boundary conditions, *Numer. Methods Partial Differential Equations* 27 (2) (2011) 436–446.
- [23] Z.-Z. Sun, Compact difference schemes for heat equation with Neumann boundary conditions, *Numer. Methods Partial Differential Equations* 25 (6) (2009) 1320–1341.
- [24] L.-B. Liu, H.-W. Liu, A new fourth-order difference scheme for solving an N-carrier system with Neumann boundary conditions, *Int. J. Comput. Math.* 88 (16) (2011) 3553–3564.
- [25] G.-H. Gao, Z.-Z. Sun, Compact difference schemes for heat equation with Neumann boundary conditions (II), *Numer. Methods Partial Differential Equations* 29 (5) (2013) 1459–1486.
- [26] A. Wiegmann, Fast Poisson, Fast Helmholtz and Fast Linear Electrostatic Solvers on Rectangular Parallelepipeds, Lawrence Berkeley National Laboratory, 1999.
- [27] X. Feng, H.-J. Wu, A posteriori error estimates and an adaptive finite element method for the Allen-Cahn equation and the mean curvature flow, *J. Sci. Comput.* 24 (2) (2005) 121–146.